

复旦大学

本科生课程实验报告

实验题目:	Lab1 Data Lab
修读课程:	计算机系统基础
姓 名:	雍畅
学 号:	25800680018
完成日期:	2026年10月9日

摘要

这次Data Lab实验要求在指定运算规则和操作数限制内，完成十九个与32位整数和单精度浮点数有关的函数。我按题目顺序记录了实现思路、关键表达式和边界情况，重点分析了溢出判断与最近偶数舍入。

完成代码后，我先检查每题的运算规则，再重新编译并运行完整测试。十九题均通过规则检查；本地测试和GitHub Actions自动评分都是110/110。

整数题让我进一步理解了位模式和数学结果的区别；浮点题则让我学会先确定有效数和丢弃位，再决定是否进位。报告后半部分给出测试结果和具体的边界分析。

关键词：Data Lab；补码；位运算；溢出；最近偶数舍入

目录

一、实验任务与环境.....	1
1.实验环境与源码版本.....	1
2.实验文件与准备过程.....	1
二、机器模型与设计依据.....	3
1.补码与位向量模型.....	3
2.单精度编码与最近偶数舍入.....	3
三、十九个函数的设计与实现.....	4
P1 signMask 最高位掩码.....	4
P2 bitXor 受限运算实现异或.....	4
P3 negativePart 负数取相反数与非负数归零.....	4
P4 copyByteWithin 在同一字内复制字节.....	5
P5 logicalShift 由算术右移实现逻辑右移.....	5
P6 swapNibblePairs 交换每字节内的两个半字节.....	5
P7 secondLowestZeroBit 定位第二个最低零位.....	6
P8 oddParity 计算附加的奇校验位.....	6
P9 rotateRightBits 循环右移任意非负位数.....	7
P10 roundEvenPow2 舍入到二的幂次倍数.....	7
P11 midpointTowardFirst 向第一参数取整的中点.....	8
P12 isBetweenEitherOrder 判断无序端点闭区间.....	8
P13 mul5Sat 五倍乘法的饱和计算.....	9
P14 classifyAdd3 对三项精确和分类.....	10
P15 floatScaleThreeHalves 单精度浮点数乘以3/2.....	10
P16 floatRoundEven 保留浮点编码的最近偶整数舍入.....	11
P17 float_i2f 整数转换为单精度编码.....	12
P18 bitCount 并行统计置位数量.....	13
P19 bitReverse 分层交换实现整字位逆序.....	14
四、测试方法与实验结果.....	15

1.逐题检查与边界覆盖	15
2.十九题结果记录	15
3.完整检查与测试结果	16
4.边界测试与复测	18
五、代码提交与自动评分	19
六、分析与总结	21
1.位向量运算与无分支选择	21
2.有限字长下的数值处理	21
3.实验验证与个人收获	22
参考资料	23

一、实验任务与环境

我在课程Data Lab模板的bits.c中完成了十九个函数。整数题主要用受限位运算处理32位数据，浮点题则用整数运算模拟单精度编码，各题还须遵守规定的合法运算符与操作数上限。下面依次说明实现思路、测试过程和实验收获。

1.实验环境与源码版本

主机系统：Windows 11，版本10.0.26200.9457；WSL：WSL 2，版本2.7.14.0；Linux发行版：Ubuntu 22.04.5 LTS；处理器架构：x86-64。GCC版本为11.4.0，GNU Make为4.3，Python为3.10.12，Git为2.34.1；编辑器为连接到WSL Ubuntu-22.04的Visual Studio Code。

实验在2026年10月9日完成。个人仓库地址为<https://github.com/KlineCY/ics-lab1>，本地目录为/home/yonchn/ics/lab1-datalab。

我按照课程Makefile在本地使用-O -Wall -fwrapv -m32编译，测试程序为32位目标。实际构建命令见图5。

```
yonchn@YonChn:/mnt/c/Users/YonChn$ uname -m
x86_64
yonchn@YonChn:/mnt/c/Users/YonChn$ cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
yonchn@YonChn:/mnt/c/Users/YonChn$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
yonchn@YonChn:/mnt/c/Users/YonChn$ make --version
GNU Make 4.3
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
yonchn@YonChn:/mnt/c/Users/YonChn$ python3 --version
Python 3.10.12
yonchn@YonChn:/mnt/c/Users/YonChn$ git --version
git version 2.34.1
yonchn@YonChn:/mnt/c/Users/YonChn$ |
```

图1 实验环境与工具版本

2.实验文件与准备过程

bits.c保存十九个函数的实现，bits.h提供函数声明。check_ops.py封装dlc并按当前题目检查规则，decl.c提供题目输入范围与分值，tests.c提供参考行为，btest负责正确性测试。Makefile生成btest、ishow和fshow，test.sh串联构建、规则检查及完整测试。

我从课程模板创建个人仓库，并克隆到/home/yonchn/ics/lab1-datalab。初始模板可以构建，但占位代码的btest得分为0/110；之后在bits.c中逐题完成实现。

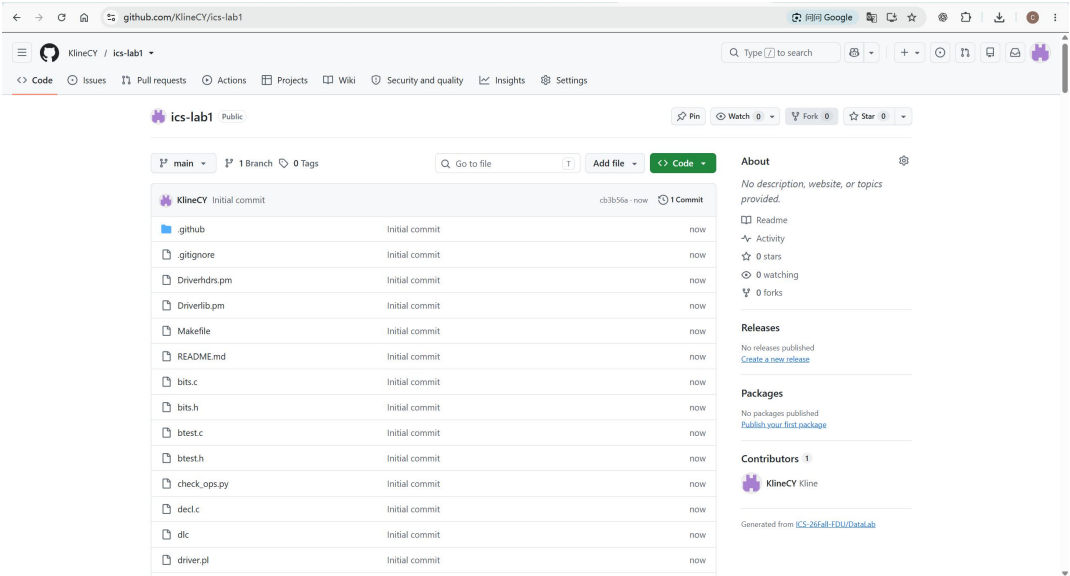


图2 个人仓库主页

二、机器模型与设计依据

1. 补码与位向量模型

本实验按课程约定的32位补码位向量模型分析int：算术右移复制符号位，加法和左移保留低32位，移位量限定在0至31之间。该约定用于解释本实验的位运算，不能直接视为标准C对有符号溢出和左移行为的一般保证。

$$INT_MIN = -2^{31}; \quad INT_MAX = 2^{31} - 1$$

整数题仅允许int参数与局部变量、0至255的常量，以及!、~、&、^、|、+、<<、>>。P2进一步限定为~和&。P18与P19虽排在浮点题之后，仍遵循整数题规则。表达式赋值本身不计操作数，但构造掩码、取反、移位和组合结果的运算都计入预算。

符号掩码 $x \gg 31$ 在负数时为全一，在非负数时为全零。对于全字掩码m，可用 $(a \& m) | (b \& \sim m)$ 选择a或b；若条件只取0或1，则先把它扩展成全字掩码。符号相同的两数可以通过差值符号比较，符号不同的两数则应先使用符号关系，避免差值溢出影响判断。

2. 单精度编码与最近偶数舍入

单精度编码由1位符号s、8位偏置阶码E和23位小数字段F组成。规格化数具有隐含的首位1；非规格化数没有这一位，并使用固定的最小指数。E为255时，F为零表示无穷，F非零表示NaN。

$$v = (-1)^s \times (1 + F \times 2^{-23}) \times 2^{E-127} \quad (1 \leq E \leq 254)$$

$$v = (-1)^s \times F \times 2^{-149} \quad (E = 0)$$

P15至P17允许int与unsigned、整数常量、整数运算和控制流，因此可以分类处理阶码与舍入；实现仍不能使用浮点类型、浮点运算、类型转换或函数调用。返回unsigned只是承载浮点编码，并不意味着返回浮点数的整数数值。

最近偶数舍入先比较丢弃部分与半程：小于半程保留，大于半程进位；恰好半程时，根据保留部分的最低位决定是否进位，使最终最低位为偶数。以下统一用q表示保留部分、r表示丢弃部分，丢弃位数为k。

$$h = 2^{k-1}; \quad \text{增量} = 1 \Leftrightarrow r > h \text{ 或 } (r = h \text{ 且 } q \text{ 为奇数})$$

最近偶数规则分别作用于P10的整数商、P15的有效数、P16的整数部分和P17的24位有效数。实现前要确定保留位数，并考虑进位是否改变阶码。后文主要围绕三类问题展开：用掩码重组位，用有限字长实现整数运算，以及用整数运算处理浮点编码。

三、十九个函数的设计与实现

下面按P1至P19的顺序介绍每题的代码、主要思路和代表输入。各题实际操作数和测试得分集中列在第四部分。

P1 signMask 最高位掩码

本题分值为1分，操作数上限为2，采用整数题规则。

将1左移31位，即可得到仅最高位为一的32位字。结果为0x80000000，按有符号整数解释时是INT_MIN。这里构造的是位模式，表达式只用一次移位。

```
int signMask(void) {  
    return 1 << 31;  
}
```

预期结果：signMask()返回0x80000000。查看结果时同时注意十六进制位模式和有符号十进制显示。

P2 bitXor 受限运算实现异或

本题分值为2分，操作数上限为8，仅允许~与&。

异或在两输入位不同时取一。 $\sim(\sim x \ \& \ \sim y)$ 等价于 $x \mid y$ ，表示至少有一位为一； $\sim(x \ \& \ y)$ 排除两位同为一的情况。将二者相与，就能只用~和&实现异或。

```
int bitXor(int x, int y) {  
    return  $\sim(\sim x \ \& \ \sim y) \ \& \ \sim(x \ \& \ y)$ ;  
}
```

预期结果：bitXor(4,5)=1；bitXor(7,7)=0；bitXor(0,0x80000000)=0x80000000。相同输入、单比特差异和符号位差异分别覆盖零结果、一般异或和最高位。

P3 negativePart 负数取相反数与非负数归零

本题分值为3分，操作数上限为6，采用整数题规则。

$x \gg 31$ 将负数的符号扩展为全一，将非负数扩展为全零。用这个掩码与 $\sim x + 1$ 相与，就能对负数取相反数、对非负数返回零。INT_MIN的数学相反数超出32位有符号范围，按本实验的回绕规则仍返回0x80000000。

```
int negativePart(int x){  
    int mask =  $x \gg 31$ ;  
    return  $(\sim x + 1) \ \& \ mask$ ;  
}
```

预期结果：negativePart(-10)=10；negativePart(5)=0；negativePart(0)=0；negativePart(0x80000000)=0x80000000。

P4 copyByteWithin 在同一字内复制字节

本题分值为4分，操作数上限为12，采用整数题规则。输入范围为 $0 \leq \text{src} \leq 3$ 、 $0 \leq \text{dst} \leq 3$ 。

字节编号从最低字节0到最高字节3。 $\text{src} \ll 3$ 和 $\text{dst} \ll 3$ 分别得到源、目标偏移。先右移并用0xff提取源字节，再清空目标八位并写入源字节，其余位置保持原值。源值先从原输入提取，因此src与dst相同时结果不变。

```
int copyByteWithin(int x, int src, int dst) {
    int s = src << 3;
    int d = dst << 3;
    int byte = (x >> s) & 0xff;
    int mask = 0xff << d;
    return (x & ~mask) | (byte << d);
}
```

预期结果：copyByteWithin(0x11223344,0,2)=0x11443344;
copyByteWithin(0x11223344,3,3)=0x11223344;
copyByteWithin(0x80223344,3,0)=0x80223380。

P5 logicalShift 由算术右移实现逻辑右移

本题分值为4分，操作数上限为20，采用整数题规则。输入范围为 $0 \leq n \leq 31$ 。

$x \gg n$ 会对负数进行符号扩展，因此还要用掩码只保留低 $32-n$ 位。掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ 在 $n=0$ 时为全一，在 $n=31$ 时为1，两个端点都正确，并且不会执行移位32位的操作。

```
int logicalShift(int x, int n) {
    int mask = ~(((1 << 31) >> n) << 1);
    return (x >> n) & mask;
}
```

预期结果：logicalShift(0x87654321,4)=0x08765432; logicalShift(0x80000000,31)=1;
logicalShift(0x80000000,0)=0x80000000。

P6 swapNibblePairs 交换每字节内的两个半字节

本题分值为4分，操作数上限为18，采用整数题规则。

先用0x0f构造0x0f0f0f0f。每字节的低四位经掩码保留后左移四位，高四位则右移四位后再取掩码，最后按位或合并。掩码会清除算术右移的符号扩展与跨字节内容，使交换只发生在各字节内部。

```
int swapNibblePairs(int x) {
    int mask = 0x0f | (0x0f << 8);
    mask = mask | (mask << 16);
    return ((x & mask) << 4) | ((x >> 4) & mask);
}
```

预期结果：swapNibblePairs(0x12345678)=0x21436587;
swapNibblePairs(0x000000ab)=0x000000ba; 对任一输入连续执行两次应恢复原位模式。

P7 secondLowestZeroBit 定位第二个最低零位

本题分值为4分，操作数上限为8，采用整数题规则。

先对x取反，把零位变成一位。zeros & (zeros + ~0)清除最低的一位，再用rest & (~rest + 1)取出剩余最低的一位，即原输入的第二个最低零位。若原输入不足两个零位，rest为零，结果也为零。返回值是单比特掩码，而非位置编号。

```
int secondLowestZeroBit(int x) {  
    int zeros = ~x;  
    int rest = zeros & (zeros + ~0);  
    return rest & (~rest + 1);  
}
```

预期结果：secondLowestZeroBit(0xffffffff)=0x4; secondLowestZeroBit(0x7fffffff)=0;
secondLowestZeroBit(0xffffffff)=0; secondLowestZeroBit(0)=2。

P8 oddParity 计算附加的奇校验位

本题分值为5分，操作数上限为56，采用整数题规则。

依次右移16、8、4、2、1位并异或，把32位中一的数量奇偶性归并到最低位。将该位与1异或，就得到附加的奇校验位：原输入有偶数个一时返回1，有奇数个一时返回0。算术右移引入的高位不影响最终最低位。

```
int oddParity(int x) {  
    x = x ^ (x >> 16);  
    x = x ^ (x >> 8);  
    x = x ^ (x >> 4);  
    x = x ^ (x >> 2);  
    x = x ^ (x >> 1);  
    return (x & 1) ^ 1;  
}
```

预期结果：oddParity(5)=1; oddParity(7)=0; oddParity(0)=1; oddParity(0xffffffff)=1。最后两个输入的一的数量分别为0和32，均为偶数。

P9 rotateRightBits 循环右移任意非负位数

本题分值为5分，操作数上限为16，采用整数题规则。 n 为非负int，实际循环位数为 n 对32取余。

循环移位以32为周期，因此先用 $n \& 31$ 取得实际位数 k 。右侧部分沿用逻辑右移的掩码构造；左侧部分把被移出的低位送至高位，再按位或组合。左移位数写成 $(32-k)$ 模32，并通过补码加法与掩码实现。

$k=0$ 时，两部分都等于 x ，按位或仍为 x ；左移量经过 $\&31$ 后为零，不会执行移位32位。 $k=31$ 时左移一位，右侧仅保留原最高位。因为输入 n 保证非负，按低五位归约与除以32取余一致。

```
int rotateRightBits(int x, int n) {
    int k = n & 31;
    int right = (x >> k) & ~(((1 << 31) >> k) << 1);
    int left = x << ((32 + (~k + 1)) & 31);
    return right | left;
}
```

预期结果：rotateRightBits(0x12345678,8)=0x78123456；位数0与32均保持输入；
rotateRightBits(1,31)=2。

P10 roundEvenPow2 舍入到二的幂次倍数

本题分值为5分，操作数上限为24，采用整数题规则。输入范围为 $0 \leq x \leq 0x3ffffff$ 、 $1 \leq n \leq 16$ 。

设 x 由商 q 和余数 r 组成，除数为2的 n 次幂。小于半程向下舍入，大于半程向上舍入；恰好半程时选择偶数商。实现给 x 加上“半程减一再加商最低位”的偏置，随后清除最低 n 位。

当商为偶数时，偏置为半程减一，余数恰好半程也不足以进位；当商为奇数时，偏置为半程，恰好半程会进到下一偶数商。输入上界为 $0x3ffffff$ ， n 不超过16，加入偏置不会达到有符号正数上界，因此本题不需要饱和处理。

$$x = q \times 2^n + r; \quad b = 2^{n-1} - 1 + (q \bmod 2)$$

```
int roundEvenPow2(int x, int n) {
    int half = 1 << (n + ~0);
    int bias = (half + ~0) + ((x >> n) & 1);
    int mask = (1 << n) + ~0;
    return (x + bias) & ~mask;
}
```

预期结果：roundEvenPow2(10,2)=8；roundEvenPow2(14,2)=16；
roundEvenPow2(11,2)=12；roundEvenPow2(12,2)=12。前两例分别对应偶商与奇商的中点。

P11 midpointTowardFirst 向第一参数取整的中点

本题分值为5分，操作数上限为32，采用整数题规则。

直接计算 $x+y$ 可能溢出。恒等式 $x+y=2(x \& y)+(x \wedge y)$ 把相加拆成公共位与不同位；因此 $(x \& y)+((x \wedge y)>>1)$ 给出向下取整的数学中点。算术右移保证负奇数也按向负无穷方向取整。

两输入之和为奇数时，中点位于相邻整数之间。若 x 较大，应给向下取整值加一；若 x 较小，则保留原值。实现从 $(x \wedge y)\&1$ 提取奇偶信息，再按输入是否异号构造大小关系：异号时正数更大，同号时差值不溢出，可使用差值符号。变量`greater`在相等时也取一，但此时`odd`为零，不会错误加一。

$$\lfloor (x + y) / 2 \rfloor = (x \& y) + ((x \wedge y) >> 1)$$

```
int midpointTowardFirst(int x, int y) {
    int floorMid = (x & y) + ((x ^ y) >> 1);
    int odd = (x ^ y) & 1;
    int signDiff = (x ^ y) >> 31;
    int diff = x + (~y + 1);
    int greater = ((~signDiff & ~(diff >> 31)) | (signDiff & ~(x >> 31))) & 1;
    return floorMid + (odd & greater);
}
```

预期结果：`midpointTowardFirst(4,7)=5`，交换参数得到6；`midpointTowardFirst(-4,-7)=-5`。表2通过交换两个极值参数，检查向第一参数取整的方向。

P12 isBetweenEitherOrder 判断无序端点闭区间

本题分值为7分，操作数上限为48，采用整数题规则。

分别计算 x 是否大于等于 a ，以及 x 是否大于等于 b 。端点不相等且 x 不等于端点时，处于区间内部意味着两个比较结果恰有一个为真。对两个结果异或，再显式纳入 x 等于 a 或 b 的情况，就得到闭区间判定。

比较先检查符号是否不同。异号时仅看 x 的符号；同号时再看 x -端点的符号，因为同号32位整数之差处于可表示范围。端点相等时，两个比较结果相同，只有显式相等条件能返回真，从而正确退化为单点集合。

```
int isBetweenEitherOrder(int x, int a, int b) {
    int sa = (x ^ a) >> 31;
    int sb = (x ^ b) >> 31;
    int da = x + (~a + 1);
    int db = x + (~b + 1);
    int gea = ((sa & ~(x >> 31)) | (~sa & ~(da >> 31))) & 1;
    int geb = ((sb & ~(x >> 31)) | (~sb & ~(db >> 31))) & 1;
    return (gea ^ geb) | !(x ^ a) | !(x ^ b);
}
```

预期结果: isBetweenEitherOrder(5,8,3)=1; isBetweenEitherOrder(3,8,3)=1;
isBetweenEitherOrder(9,8,3)=0; 区间[INT_MIN,INT_MAX]包含0; 端点均为3时只接受
x=3。

P13 mul5Sat 五倍乘法的饱和计算

本题分值为7分, 操作数上限为30, 采用整数题规则。

五倍结果能精确表示时, 5x必须位于有符号32位范围内。由于x为整数, 可以得到以下输入范围。实现用四个小常量拼出limit=0x19999999, 即429496729。

$$-2^{31} \leq 5x \leq 2^{31} - 1 \quad \Leftrightarrow \quad -429496729 \leq x \leq 429496729$$

代码同时计算hi和lo, 再由符号掩码选择有效判定。非负输入采用x-limit-1的符号, 负输入采用x+limit的符号; 各自有效的差值都在可表示范围内。另一项即使回绕, 也被掩码排除。over为全一时选择同号饱和值, 否则保留(x<<2)+x。先判断输入阈值, 可以避免五倍结果回绕后符号不变而漏判溢出。

```
int mul5Sat(int x) {  
    int limit = (0x19 << 24) | (0x99 << 16) | (0x99 << 8) | 0x99;  
    int hi = (x + ~limit) >> 31;  
    int lo = (x + limit) >> 31;  
    int sx = x >> 31;  
    int over = (~sx & ~hi) | (sx & lo);  
    int value = (x << 2) + x;  
    int sat = sx ^ ~(1 << 31);  
    return (value & ~over) | (sat & over);  
}
```

表2列出正负饱和阈值内外的实测结果, 分别检查精确乘积与INT_MAX、INT_MIN的选择。

P14 classifyAdd3 对三项精确和分类

本题分值为7分，操作数上限为52，采用整数题规则。

本题判断精确数学和 $x+y+z$ 相对有符号32位范围的位置。设 s 、 t 分别为两次加法的有符号32位回绕结果， c_1 、 c_2 为每步的高位修正：正溢出取1，负溢出取-1，无溢出取0。两次加法分别满足：

$$x + y = s + c_1 2^{32}, \quad s + z = t + c_2 2^{32}$$

把第一式代入第二式，可得三项精确和与最终回绕结果的关系。代码用 $ov1$ 、 $ov2$ 检测同号输入相加得到异号结果的情况，再用相应输入的符号确定修正方向。

$$x + y + z = t + (c_1 + c_2) 2^{32}, \quad high = c_1 + c_2$$

```
int classifyAdd3(int x, int y, int z) {
    int s = x + y;
    int t = s + z;
    int ov1 = (~x ^ y) & (x ^ s) >> 31;
    int ov2 = (~s ^ z) & (s ^ t) >> 31;
    int high = (ov1 & (1 | (x >> 31))) + (ov2 & (1 | (s >> 31)));
    return (high >> 31) | !!high;
}
```

t 本身位于有符号32位范围内，因此 $high$ 的正、负、零分别对应上溢、下溢和可表示。两次相反方向的修正可能抵消，不能仅凭中间溢出就作最终分类。表2给出了这种情况的实测结果：单向越界时，`classifyAdd3(INT_MAX,1,0)`应返回1，`classifyAdd3(INT_MIN,-1,0)`应返回-1。

P15 floatScaleThreeHalves 单精度浮点数乘以3/2

本题分值为7分，操作数上限为60，采用浮点题规则。

先拆出符号、阶码和尾数字段。零、无穷与NaN直接返回原位模式；其余输入把有效数作为整数处理。规格化数补上隐藏的一，非规格化数直接使用尾数字段。三倍有效数由 $sig+(sig<<1)$ 精确得到，最大值仍在`unsigned`范围内。

结果为原数的1.5倍。若规格化数的三倍有效数达到2的25次幂，则右移两位并使阶码增加一；否则右移一位。非规格化数以最小间距为单位，仅右移一位。统一保存丢弃部分`rem`，并以大于半程或等于半程且保留值为奇数的条件进位，从而只在最终精度执行一次舍入。

```

unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exp = (uf >> 23) & 0xffu;
    unsigned frac = uf & 0x7ffffu;
    unsigned sig, triple, shift, q, rem, half, outExp;
    if (exp == 0xffu || (exp == 0 && frac == 0)) return uf;
    sig = exp ? ((1u << 23) | frac) : frac;
    triple = sig + (sig << 1);
    shift = (exp && triple >= (1u << 25)) ? 2u : 1u;
    q = triple >> shift;
    rem = triple & ((1u << shift) - 1u);
    half = 1u << (shift - 1u);
    if (rem > half || (rem == half && (q & 1u))) q++;
    outExp = exp + (shift == 2u);
    if (exp && q >= (1u << 24)) {
        q >>= 1;
        outExp++;
    }
    if (!exp && q >= (1u << 23)) outExp = 1u;
    if (outExp >= 0xffu) return sign | 0x7f800000u;
    if (outExp == 0) return sign | q;
    return sign | (outExp << 23) | (q & 0x7ffffu);
}

```

以最大正非规格化数0x007fffff为例， $\text{sig}=2^{23}-1$ ，三倍有效数为25165821。除以2得到12582910.5，恰好处于中点，保留偶数12582910。该值达到 2^{23} ，已进入规格化区间，因此outExp由0变为1；去掉隐藏位后的小数字段为0x3ffffe，最终编码为0x00bffffe。表2记录了这个输入的实测结果。

其他预期结果：0x00000001变为0x00000002，0x00000003变为0x00000004，0x3f800000变为0x3fc00000；阶码溢出时输出同号无穷，负零与NaN原样保留。

P16 floatRoundEven 保留浮点编码的最近偶整数舍入

本题分值为10分，操作数上限为65，采用浮点题规则。

返回值仍是浮点位模式。由偏置阶码exp确定小数位数量：exp小于126时绝对值小于0.5，返回带原符号的零；exp等于126时数值在0.5至1之间，恰为0.5取零，其余取同号1；exp达到150时二进制间距至少为1，输入已无小数位。

对exp在127至149的输入，最低 $k=150-\text{exp}$ 位属于小数部分。提取rem并清空这k位，得到向零截断的浮点编码。超过半程时进位；恰好半程时，仅当保留整数为奇数才进位。当exp=127时，输入绝对值在[1,2)内，截断后的绝对值为1。此时 $k=23$ ， $(\text{uf} \gg k) \& 1\text{u}$ 读取的是阶码最低位，恰好为1，与保留整数的奇偶性相符。因此，正负中点1.5和-1.5分别舍入为2.0和-2.0；隐藏位本身并未存储在第23位。

```

unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exp = (uf >> 23) & 0xffu;
    unsigned frac = uf & 0x7ffffu;
    unsigned k, mask, rem, half, result;
    if (exp == 0xffu || exp >= 150u) return uf;
    if (exp < 126u) return sign;
    if (exp == 126u) return frac ? (sign | 0x3f800000u) : sign;
    k = 150u - exp;
    mask = (1u << k) - 1u;
    rem = frac & mask;
    half = 1u << (k - 1u);
    result = uf & ~mask;
    if (rem > half || (rem == half && ((uf >> k) & 1u))) result += 1u << k;
    return result;
}

```

编码中增加 $1 \ll k$ 会使绝对值进到相邻整数；负数仍保留符号，故同一流程可以处理正负两侧。

预期结果：0.5→+0，-0.5→-0，1.5→2.0，2.5→2.0，-1.5→-2.0；无穷与NaN保持原编码。

P17 float_i2f 整数转换为单精度编码

本题分值为10分，操作数上限为40，采用浮点题规则。

先处理零，并用unsigned保存输入位模式。负数幅值通过无符号取反加一得到，从而使INT_MIN的幅值2147483648仍可表示。不断右移副本得到最高有效位编号msb，浮点偏置阶码为msb+127。

msb不超过23时，幅值最多含24个有效位，可以通过左移精确对齐。否则保留最高24位，并记录被丢弃的低位。舍入条件与前两道浮点题一致；进位后若有效数达到2的24次幂，则右移一位并增加阶码。最后去掉隐藏位，再组合符号、阶码和小数字段。

```

unsigned float_i2f(int x) {
    unsigned ux = x;
    unsigned sign = x < 0 ? 0x80000000u : 0u;
    unsigned mag = x < 0 ? ~ux + 1u : ux;
    unsigned temp = mag;
    unsigned msb = 0u;
    unsigned sig, shift, rem, half;
    if (x == 0) return 0u;
    while (temp >> 1) {
        temp >>= 1;
        msb++;
    }
    if (msb <= 23u) {
        sig = mag << (23u - msb);
    } else {
        shift = msb - 23u;
        sig = mag >> shift;
        rem = mag & ((1u << shift) - 1u);
        half = 1u << (shift - 1u);
        if (rem > half || (rem == half && (sig & 1u))) sig++;
        if (sig == (1u << 24)) {
            sig >>= 1;
            msb++;
        }
    }
    return sign | ((msb + 127u) << 23) | (sig & 0x7ffffu);
}

```

预期结果：0→0x00000000, 1→0x3f800000, -1→0xbf800000,

INT_MIN→0xc0000000, INT_MAX舍入为0x4f000000。表2选取两个相邻的舍入中点，检查保留部分为偶数和奇数时的不同选择。

P18 bitCount 并行统计置位数量

本题分值为10分，操作数上限为40，采用整数题规则。

前三层依次把组宽扩展为2位、4位和8位，并保持每组保存原始对应位中一的数量这一性质。计数上限分别为2、4和8，均能放入对应字段；掩码只保留组内所需的部分，因此分组求和不会混入相邻组的信息。

随后右移8位、16位相加，逐步把相邻字节的计数累加到最低字节。这两步保留了其他位置的部分和，但最终只取最低六位；总计数最多32，不会从最低字节向外进位。整个算法通过固定层数完成归并。

```

int bitCount(int x) {
    int m1 = 0x55 | (0x55 << 8);
    int m2 = 0x33 | (0x33 << 8);
    int m4 = 0x0f | (0x0f << 8);
    m1 = m1 | (m1 << 16);
    m2 = m2 | (m2 << 16);
    m4 = m4 | (m4 << 16);
    x = (x & m1) + ((x >> 1) & m1);
    x = (x & m2) + ((x >> 2) & m2);
    x = (x + (x >> 4)) & m4;
    x = x + (x >> 8);
    x = x + (x >> 16);
    return x & 0x3f;
}

```

预期结果：bitCount(0)=0；bitCount(0xffffffff)=32；bitCount(0x80000000)=1；bitCount(0x55555555)=16。全一输入用于检查最大计数和分组间进位。

P19 bitReverse 分层交换实现整字位逆序

本题分值为10分，操作数上限为34，采用整数题规则。

依次交换相邻单比特、两位组、半字节、字节和半字，最终使原第*i*位到达第31-*i*位。每层将组内高半部分右移并掩码，将低半部分先掩码再左移，然后按位或合并。右移后的掩码还会清除算术右移引入的符号位。

操作数限制只有34，因此掩码构造也是设计重点。先构造0x00ff00ff，再利用与左移副本异或依次得到0x0f0f0f0f、0x33333333和0x55555555，复用层间关系。位编号可写成五位二进制，每层交换翻转其中一个编号位，五层之后编号全部取反，即31-*i*。

```

int bitReverse(int x)
{
    int m8 = 0xff | (0xff << 16);
    int m4 = m8 ^ (m8 << 4);
    int m2 = m4 ^ (m4 << 2);
    int m1 = m2 ^ (m2 << 1);
    int m16 = 0xff | (0xff << 8);
    x = ((x >> 1) & m1) | ((x & m1) << 1);
    x = ((x >> 2) & m2) | ((x & m2) << 2);
    x = ((x >> 4) & m4) | ((x & m4) << 4);
    x = ((x >> 8) & m8) | ((x & m8) << 8);
    return ((x >> 16) & m16) | (x << 16);
}

```

预期结果：bitReverse(0x80000004)=0x20000001；bitReverse(0x7fffffff)=0xffffffffe；bitReverse(0x01234567)=0xe6a2c480。连续逆序两次应恢复原值，也可用来补充检查。

四、测试方法与实验结果

1. 逐题检查与边界覆盖

我先用check_ops.py核对合法运算符和操作数，再重新构建并运行btest，与课程参考行为比较；最后通过GitHub Actions检查已推送版本的远程评分。这三步分别检查编码约束、所执行用例的正确性和远程运行结果。

代表输入主要选在容易出错的边界：整数题包括零、最高位、移位端点、符号组合和饱和阈值两侧；浮点题包括正负零、非规格化数、舍入中点、有效数进位、无穷和NaN。规则检查与完整测试结果见图3至图5。

逐题检查时，我用函数名指定目标。下面以midpointTowardFirst为例，列出实际使用的命令。

```
./check_ops.py --function midpointTowardFirst bits.c
make clean && make all
./btest -f midpointTowardFirst
./btest -f midpointTowardFirst -1 4 -2 7
./btest -f midpointTowardFirst -1 7 -2 4
```

逐题样例用于检查具体的边界条件，完整btest用于检查十九题的整体结果。测试通过说明实现通过了所执行的用例；第三部分的位运算和边界推导则解释了关键表达式为何成立。

2. 十九题结果记录

表1汇总十九题的实际操作数、本地Errors与得分，以及同一代码提交的远程得分。本地结果一栏依次列出Errors和得分。

表1 十九题操作数与测试结果

题号与函数	操作数 / 上限	本地Errors / 得分	远程得分
P1 signMask	1/2	0; 1/1	1/1
P2 bitXor	7/8	0; 2/2	2/2
P3 negativePart	4/6	0; 3/3	3/3
P4 copyByteWithin	9/12	0; 4/4	4/4
P5 logicalShift	6/20	0; 4/4	4/4
P6 swapNibblePairs	9/18	0; 4/4	4/4
P7 secondLowestZeroBit	7/8	0; 4/4	4/4
P8 oddParity	12/56	0; 5/5	5/5
P9 rotateRightBits	13/16	0; 5/5	5/5
P10 roundEvenPow2	14/24	0; 5/5	5/5
P11 midpointTowardFirst	22/32	0; 5/5	5/5
P12 isBetweenEitherOrder	35/48	0; 7/7	7/7

题号与函数	操作数 / 上限	本地Errors / 得分	远程得分
P13 mul5Sat	26/30	0; 7/7	7/7
P14 classifyAdd3	23/52	0; 7/7	7/7
P15 floatScaleThreeHalves	47/60	0; 7/7	7/7
P16 floatRoundEven	26/65	0; 10/10	10/10
P17 float_i2f	33/40	0; 10/10	10/10
P18 bitCount	28/40	0; 10/10	10/10
P19 bitReverse	34/34	0; 10/10	10/10
合计	—	0; 110/110	110/110

3.完整检查与测试结果

```
./check_ops.py bits.c
make clean && make all
./btest
./test.sh
```

check_ops.py对十九题的规则检查全部通过；完整btest中，各题Errors均为0，总分110/110。test.sh最后显示“all checks passed: 110/110”，退出状态为0。

```
yonchn@YonChn:~/ics/lab1-datalab$ ./check_ops.py bits.c | tee ~/ics/lab1-records/final-ops.txt
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 14/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 35/48 operators
mul5Sat: 26/30 operators
classifyAdd3: 23/52 operators
floatScaleThreeHalves: 47/60 operators
floatRoundEven: 26/65 operators
float_i2f: 33/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
yonchn@YonChn:~/ics/lab1-datalab$
```

图3 十九题完整规则检查

```

● yonchn@YonChn:~/ics/lab1-datalab$ ./btest | tee ~/ics/lab1-records/final-btest.txt
Score Rating Errors Function
1 1 0 signMask
2 2 0 bitXor
3 3 0 negativePart
4 4 0 copyByteWithin
4 4 0 logicalShift
4 4 0 swapNibblePairs
4 4 0 secondLowestZeroBit
5 5 0 oddParity
5 5 0 rotateRightBits
5 5 0 roundEvenPow2
5 5 0 midpointTowardFirst
7 7 0 isBetweenEitherOrder
7 7 0 mul5Sat
7 7 0 classifyAdd3
7 7 0 floatScaleThreeHalves
10 10 0 floatRoundEven
10 10 0 float_i2f
10 10 0 bitCount
10 10 0 bitReverse
Total points: 110/110

```

图4 十九题完整btest结果

```

● yonchn@YonChn:~/ics/lab1-datalab$ cat ~/ics/lab1-records/final-test.txt
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 14/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 35/48 operators
mul5Sat: 26/30 operators
classifyAdd3: 23/52 operators
floatScaleThreeHalves: 47/60 operators
floatRoundEven: 26/65 operators
float_i2f: 33/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
operator check passed
Score Rating Errors Function
1 1 0 signMask
2 2 0 bitXor
3 3 0 negativePart
4 4 0 copyByteWithin
4 4 0 logicalShift
4 4 0 swapNibblePairs
4 4 0 secondLowestZeroBit
5 5 0 oddParity
5 5 0 rotateRightBits
5 5 0 roundEvenPow2
5 5 0 midpointTowardFirst
7 7 0 isBetweenEitherOrder
7 7 0 mul5Sat
7 7 0 classifyAdd3
7 7 0 floatScaleThreeHalves
10 10 0 floatRoundEven
10 10 0 float_i2f
10 10 0 bitCount
10 10 0 bitReverse
Total points: 110/110
all checks passed: 110/110

```

图5 test.sh完整检查日志

图3、图4和图5分别展示规则检查、btest以及test.sh的结果。

4.边界测试与复测

整理报告时，我在2026年10月9日对同一代码提交补做了十组边界测试。将bits.c与课程tests.c中的对应参考函数一同编译，使用与本地实验一致的-O -Wall -fwrapv -m32选项，逐项输出函数返回值并比较。表2中的十组结果均与参考函数一致，Errors为0。

补充测试使用临时程序boundary.c：对每组输入分别调用bits.c中的待测函数和tests.c中对应的test_前缀参考函数，输出actual、reference及PASS或FAIL，最后累计不一致项数。编译和执行命令如下。

```
gcc -O -Wall -fwrapv -m32 bits.c tests.c boundary.c -lm -o boundary
./boundary
```

临时程序、对应代码副本和运行日志保存在本地“补充测试记录”目录中。日志包含运行时间、代码提交号及十组实际输出，末行为“10 cases, 0 errors”。

表2中的整数结果按有符号十进制记录，浮点结果按32位十六进制编码记录。测试覆盖极值中点、饱和阈值、溢出抵消、非规格化边界和最近偶数舍入。

表2 代表性边界输入与实测结果

题目	输入	实际返回值	参考比较
P11	INT_MIN, INT_MAX	-1	一致
P11	INT_MAX, INT_MIN	0	一致
P13	429496729	2147483645	一致
P13	429496730	2147483647	一致
P13	-429496729	-2147483645	一致
P13	-429496730	-2147483648	一致
P14	INT_MAX, 1, -1	0	一致
P15	0x007fffff	0x00bffffe	一致
P17	16777217	0x4b800000	一致
P17	16777219	0x4b800002	一致

五、代码提交与自动评分

代码保存在个人仓库<https://github.com/KlineCY/ics-lab1>，本次实现对应提交b3b9807328de1cbb4f34d545917c55040b028a46。GitHub Actions对这个提交运行了自动评分。

该次运行的链接为<https://github.com/KlineCY/ics-lab1/actions/runs/37748645809>，状态为Success，得分为110/110。图7显示运行摘要，图8显示网页上的评分标注。

```
• yonchn@YonChn:~/ics/lab1-datalab$ git commit -m "Complete Data Lab implementations"
[main b3b9807] Complete Data Lab implementations
 1 file changed, 140 insertions(+), 19 deletions(-)
• yonchn@YonChn:~/ics/lab1-datalab$ git rev-parse HEAD
b3b9807328de1cbb4f34d545917c55040b028a46
• yonchn@YonChn:~/ics/lab1-datalab$ git status -sb
## main...origin/main [ahead 1]
○ yonchn@YonChn:~/ics/lab1-datalab$
```

图6 本地代码提交记录

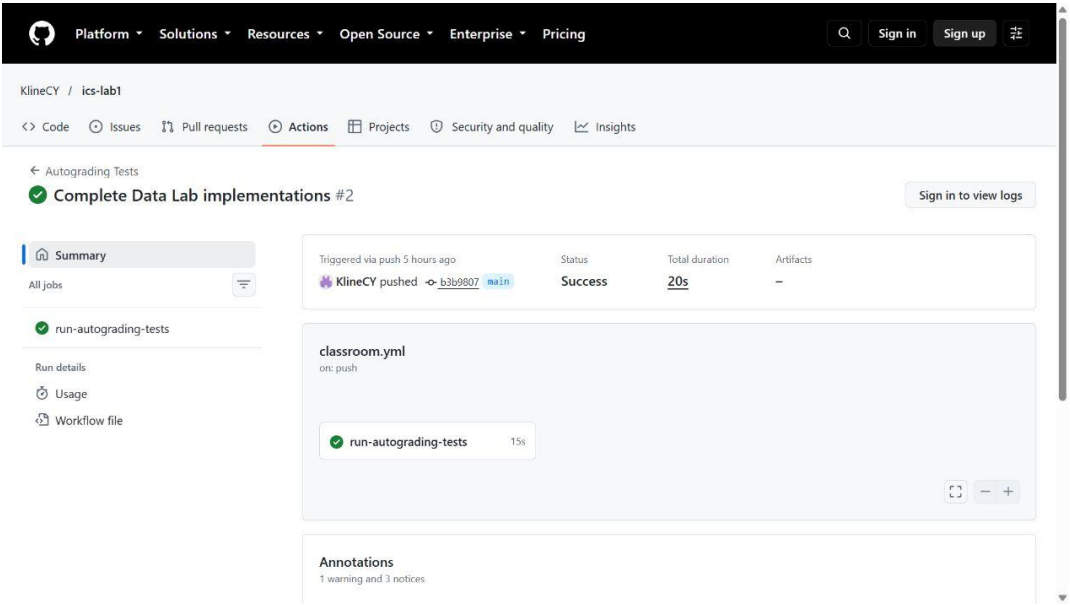


图7 GitHub Actions运行成功，关联代码提交b3b9807

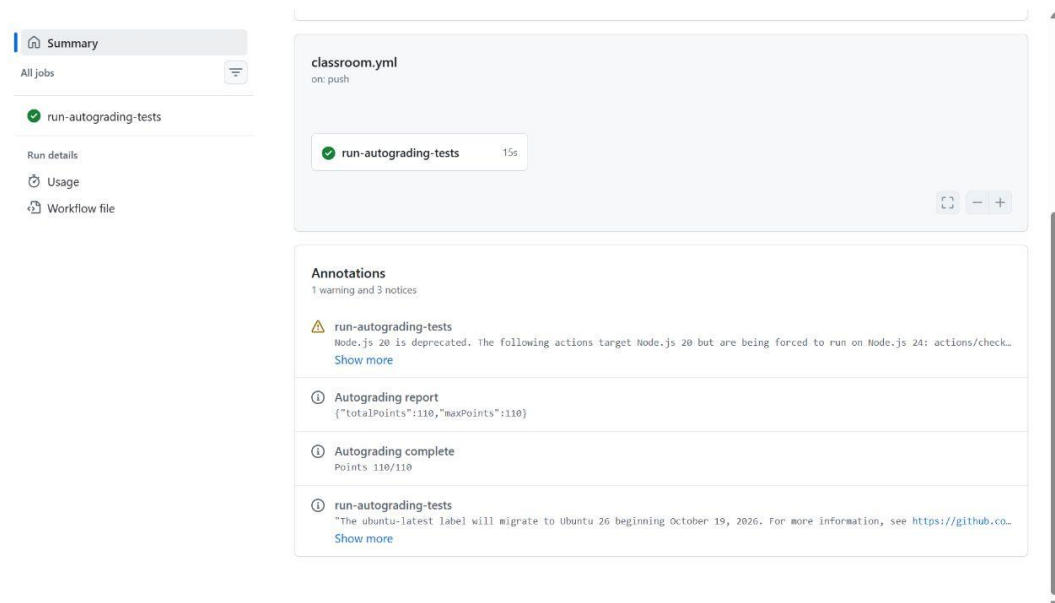


图8 GitHub Actions自动评分结果为110/110

E-Learning已提交个人仓库首页链接<https://github.com/KlineCY/ics-lab1>，提交记录见图9。

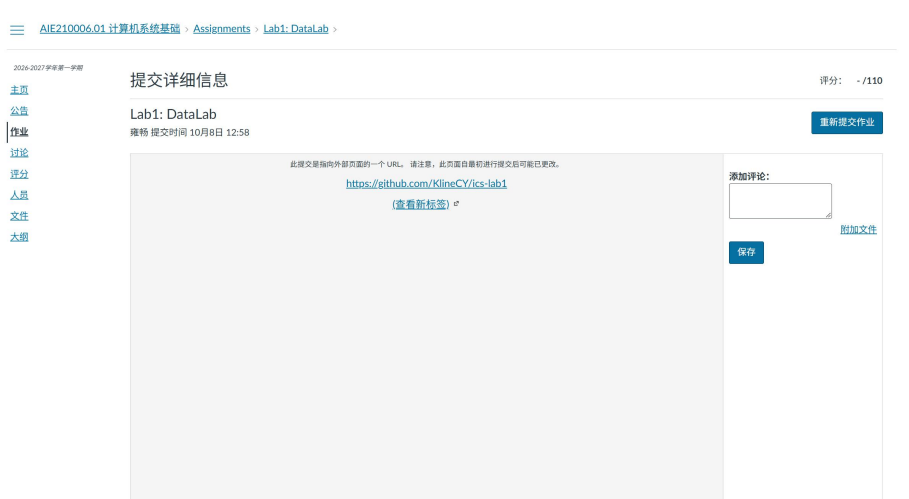


图9 E-Learning个人仓库链接提交记录

六、分析与总结

1.位向量运算与无分支选择

本实验中的位运算需要同时考虑数据的数值含义和二进制表示。对于32位补码整数，算术右移可以将符号位扩展为全零或全一掩码，从而在不使用条件语句的情况下实现数据选择。P3利用符号掩码控制负数取相反数、非负数返回零；P12则先区分比较对象是否异号，再组合相应的比较结果，避免直接相减时可能产生的溢出。两题都通过位运算实现条件判断，但前者选择整个32位结果，后者最终生成0或1的逻辑值。因此，构造表达式前必须明确掩码表示的是选择条件还是最终结果。

移位操作还需要处理符号扩展和移位边界。P5通过掩码清除算术右移引入的高位，得到逻辑右移结果；P9在此基础上将移出的低位重新送入高位，并利用对32取模的性质处理循环移位。特别是移位量为0或32的倍数时，需要避免实际执行移位32位的操作。这说明位运算表达式除了满足一般输入，还必须在机器允许的移位范围内保持正确。

P18与P19进一步体现了分组处理的思想。P18逐层合并相邻位组中的置位数量，每层保存对应分组的计数；P19则通过五层交换，使原第*i*位最终移动到第31-*i*位。两题的共同点是利用固定组宽和掩码处理整个32位字，但各层保持的信息不同：前者保存计数，后者保持位位置之间的映射关系。P19还通过复用掩码，将实现控制在34个操作数的上限内。由此可以看出，受限位运算的设计既要保证每一步的信息正确传递，也要考虑表达式的操作数成本。

2.有限字长下的数值处理

32位补码整数能够表示的范围有限，因此需要区分中间运算发生溢出与最终数学结果超出表示范围这两种情况。P11利用位运算恒等式拆分加法，在不直接计算可能溢出的 $x+y$ 的情况下求得中点，再依据中点是否为半整数及第一参数相对第二参数的大小修正取整方向。P13采用另一种处理方法：先由有符号整数范围推导出五倍乘法的安全输入区间，再通过符号掩码选择正常乘积或饱和值。这两题分别体现了避免中间溢出和处理最终结果越界的思路。

P14则需要进一步区分回绕结果与精确数学和。连续两次加法的中间结果都可能发生溢出，但最终三项和仍可能处于可表示范围。实现中分别记录两次溢出的方向，用high保存丢失的高位修正，再结合最终32位结果判断精确和的位置。例如INT_MAX+1-1的第一次加法发生正溢出，第二次发生负溢出，两次修正相互抵消，最终结果仍为INT_MAX。这个例子说明，对于连续的有限字长运算，仅凭某一步的溢出标志或最后结果的符号，无法直接判断完整数学表达式是否越界，需要保留足够的中间信息。

浮点题则在有限字长之外引入了精度限制。P10与P15—P17都涉及最近偶数舍入，其共同依据是比较丢弃部分与半程，并在恰好位于中点时依据保留部分的最低位决定是否进位。然而，各题的舍入对象和后续处理并不相同。P10直接将整数舍入到指定的二进制倍数；P15先计算三倍有效数，再进行一次舍入，避免中间舍入带来的额外精度损失；P16根

据阶码确定需要清除的小数位，并保留原有符号；P17则先确定整数的最高有效位，再保留24位有效数并完成舍入。

其中，P15的最大正非规格化数0x007fffff经过1.5倍运算后，舍入结果进入规格化区间，需要同步调整阶码；P17在处理INT_MIN时使用unsigned保存其绝对值的位模式，避免有符号取反无法表示的问题。这些边界表明，浮点运算不仅需要计算有效数，还要根据舍入后的结果重新判断编码区间、有效位长度及阶码变化。将数值运算与编码规则结合考虑，才能正确处理一般输入和特殊边界。

3.实验验证与个人收获

本实验的验证过程主要包括编码规则检查、函数正确性测试和远程自动评分。`check_ops.py`用于检查运算符种类与操作数数量，`btest`将程序结果与参考行为进行比较，GitHub Actions则验证已推送版本的运行与评分结果。十九个函数全部通过规则检查，本地测试和远程自动评分均得到110/110。需要注意的是，规则检查通过说明实现通过了课程提供的编码约束检查，测试满分说明代码通过了相应测试用例；对于关键算法的正确性，还需要结合表达式推导和边界条件进行分析。

为了进一步验证容易出错的情况，我在完成报告时补充了十组针对性测试，并与课程参考函数逐项比较。P11交换INT_MIN和INT_MAX的参数顺序，检查中点向第一参数取整的方向；P13分别测试正负饱和阈值的内外两侧；P14使用两次溢出相互抵消的输入；P15检查非规格化数进入规格化区间的情况；P17则通过两个舍入中点验证最近偶数规则。十组结果均与参考函数一致。这些测试将第三部分的关键推导与具体运行结果对应起来，也使边界处理的依据更加明确。

通过这次实验，我对整数和浮点数的机器表示有了更具体的理解。同一个32位字可以作为有符号整数、位掩码或浮点编码使用，不同解释方式对应不同的运算规则和边界条件。完成这些题目时，我逐渐形成了先明确输入范围与数学目标，再依据机器模型推导表达式，最后检查运算限制和特殊输入的处理顺序。特别是在溢出判断、舍入和分组位运算中，只有明确每一步保留了哪些信息、可能丢失哪些信息，才能解释算法为什么成立。实验也让我认识到，正确的实现需要同时满足数值语义、编码约束与测试要求，而推导和验证相互结合，能够使程序的正确性依据更加完整。

参考资料

- [1] 复旦大学计算机系统基础课程, Lab1 Data Lab实验文档, <https://ics-26fall-fdu.github.io/labs/lab1-data-lab/>。
- [2] ICS-26Fall-FDU, DataLab课程模板README, <https://github.com/ICS-26Fall-FDU/DataLab/blob/main/README.md>。
- [3] 个人仓库与GitHub Actions运行记录, <https://github.com/KlineCY/ics-lab1>;
<https://github.com/KlineCY/ics-lab1/actions/runs/37748645809>。